



DEPARTAMENTO ACADÊMICO DE ELETRÔNICA

GIULIA DE SOUZA LEITE

Projeto - Banco de Dados de uma Loja de Maquiagens

Introd. a Banco de Dados N11

Professor Danillo Leal Belmonte

CURITIBA

2026

Sumário

1. Introdução.....	3
2. Modelagem.....	4
2.1 Modelo Conceitual (DER).....	4
2.2 Modelo Lógico (IDEF1X).....	5
3. Estruturação das Entidades.....	6
3.1 Entidades.....	6
4. Implementação do Banco de Dados.....	8
4.1 Roteiro de Criação.....	8
5. População do Banco de Dados e Automação de Cálculos.....	11
5.1 Integridade e Lógica de Dados.....	11
5.2 Triggers.....	11
5.3 Scripts de Carga e Processamento.....	14
6. Consultas Analíticas e Validação do Sistema.....	18
6.1 Relatórios Comerciais.....	18
6.2 Gestão de Fornecedores.....	19
6.3. Métricas Financeiras e Auditoria de Estoque.....	20
7. Conclusão.....	22
8. Referências Bibliográficas.....	23

1. Introdução

O presente trabalho tem como objetivo o projeto e a implementação de um sistema de banco de dados relacional voltado para a gestão operacional de uma loja de maquiagens. A estrutura foi desenvolvida para atender às demandas de controle de inventário, registro de vendas e gestão de fornecedores, garantindo a integridade dos dados através de relacionamentos bem definidos entre as entidades.

O banco de dados é composto por 8 tabelas interconectadas, modeladas para suportar o ciclo completo do negócio, desde o reabastecimento de estoque até o atendimento ao consumidor final. A modelagem seguiu os princípios do padrão IDEF1X, buscando otimizar a organização das informações e facilitar a extração de métricas gerenciais, como margem de lucro, ranking de vendas e controle de estoque, essenciais para a análise de desempenho do empreendimento.

A escolha de uma arquitetura com 8 tabelas justifica-se pela necessidade de separar responsabilidades, garantindo a normalização dos dados e evitando a redundância, o que confere maior escalabilidade e segurança ao sistema em comparação com modelos mais simplificados.

Esse projeto se encontra também no [github](#).

2. Modelagem

Para a representação da arquitetura do banco de dados da loja, foram elaborados os diagramas de modelagem conceitual e lógica, essenciais para a compreensão das entidades, atributos e, sobretudo, dos relacionamentos estabelecidos entre as tabelas.

Ao contrário de abordagens puramente teóricas que utilizam exclusivamente chaves compostas em tabelas associativas, este projeto optou por incluir chaves primárias artificiais (Surrogate Keys) nestas entidades desde a fase de modelagem lógica (IDEF1X). Esta decisão visa garantir o espelhamento exato com o modelo físico e assegurar a compatibilidade nativa e a performance com frameworks de mapeamento objeto-relacional (ORMs) utilizados no desenvolvimento web, que exigem identificadores de coluna única para o gerenciamento correto da persistência de objetos.

2.1 Modelo Conceitual (DER)

O Diagrama Entidade-Relacionamento (DER) tem como finalidade ilustrar a estrutura conceitual do sistema. Ele foca em representar as entidades principais, como Clientes, Produtos e Fornecedores, e as interações entre elas (cardinalidades), abstraindo os detalhes técnicos de implementação e focando nas regras de negócio.

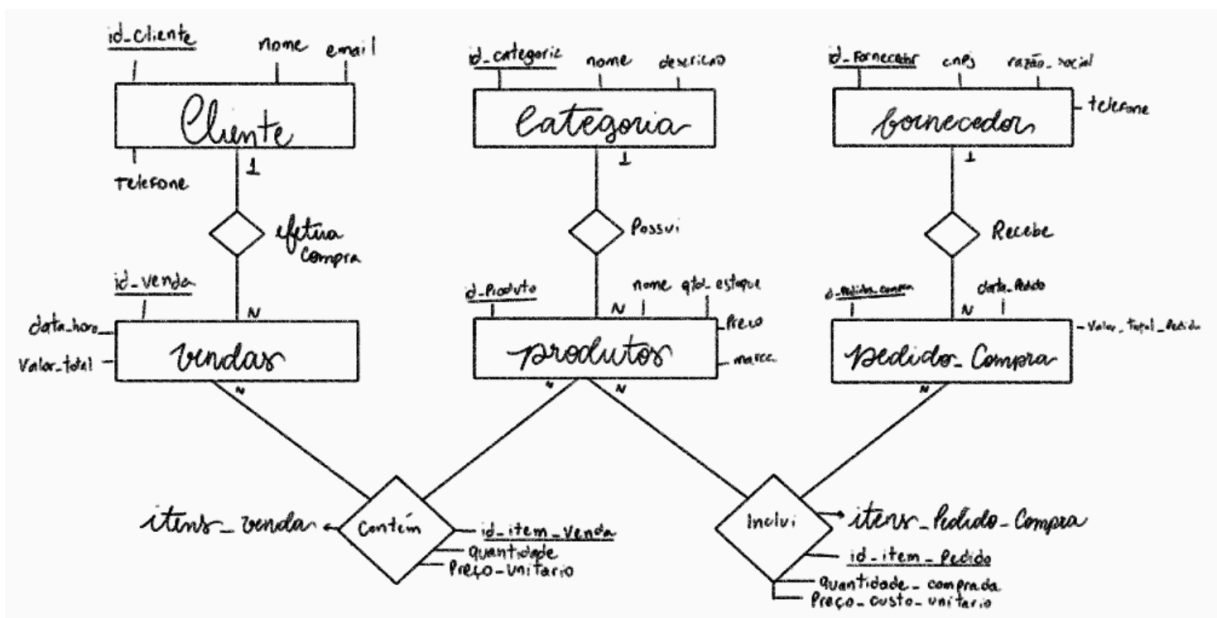


Figura 1. Diagrama Entidade-Relacionamento do sistema.

2.2 Modelo Lógico (IDEF1X)

O modelo lógico foi construído seguindo a notação IDEF1X, que é o padrão utilizado para transformar a visão conceitual em uma estrutura de banco de dados relacional. Este diagrama detalha as chaves primárias (PKs) e chaves estrangeiras (FKs) que compõem o banco, demonstrando a integridade referencial entre as 8 tabelas do sistema.

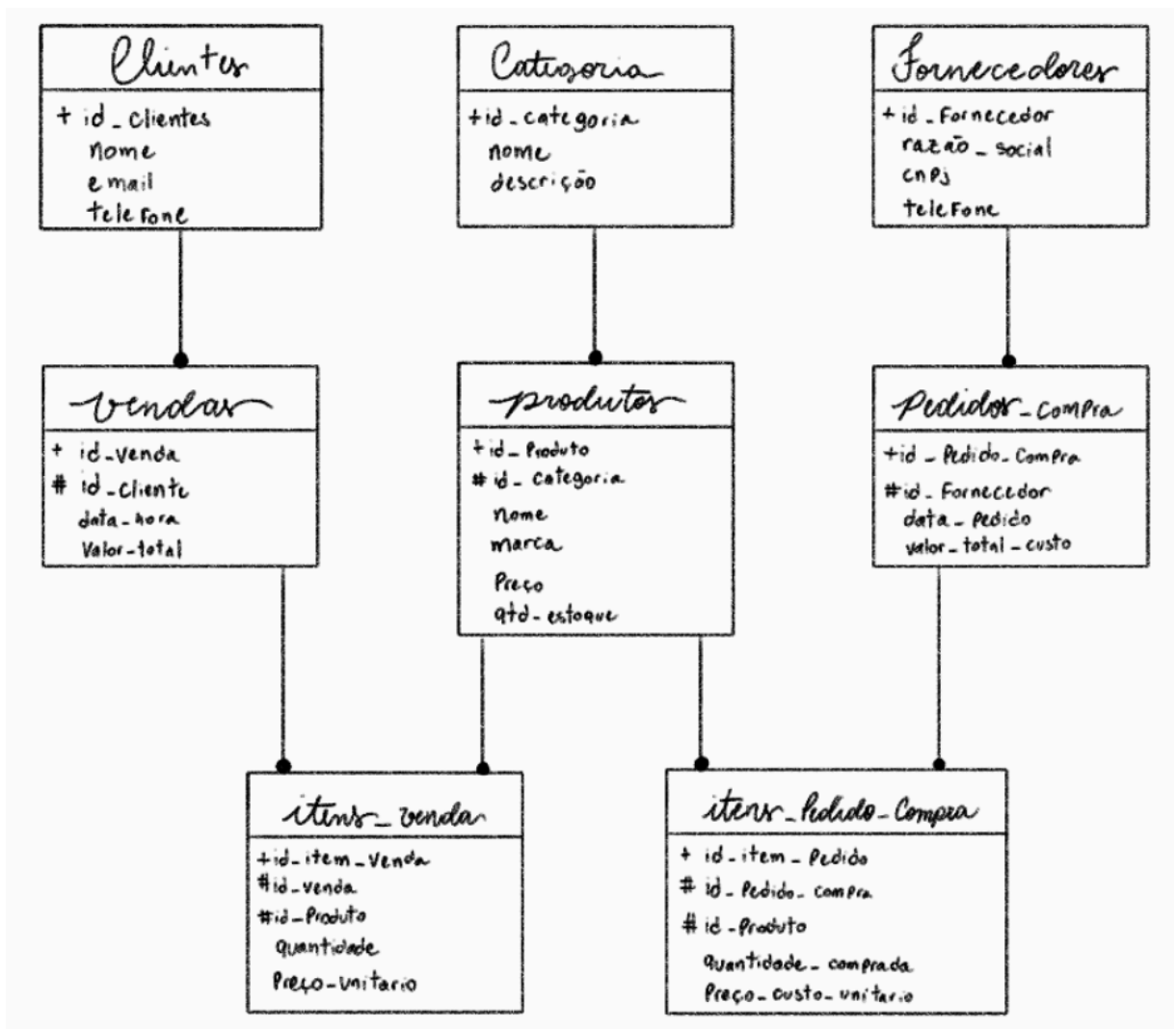


Figura 2. Modelo lógico IDEF1X do sistema .

3. Estruturação das Entidades

Na presente seção se encontra a criação de cada entidade, pensando na estrutura do banco de dados que foi concebida para permitir a rastreabilidade total de um produto, desde a entrada no estoque via fornecedor até a comercialização final para o cliente.

3.1 Entidades

As figuras a seguir ilustram a estrutura das entidades já com uma prévia dos dados popularizados para melhor visualização dos atributos.

- **CLIENTES:** Armazena os dados dos consumidores. É a base para registrar quem efetuou cada compra.

123 id_cliente	A-Z nome	A-Z email	A-Z telefone
1	Ana Silva	ana@email.com	11999999999
2	Beatriz Costa	bia@email.com	11888888888
3	Carolina Mendes	carol@email.com	11777777777
4	Daniela Souza	dani@email.com	11666666666

Figura 3. Tabela da entidade Clientes.

- **CATEGORIAS:** Organiza os produtos em grupos, facilitando a gestão do catálogo e a análise de vendas por tipo de item.

123 id_categoria	A-Z nome	A-Z descricao
1	Rosto	Bases, corretivos, pós e blushes
2	Olhos	Sombras, delineadores e máscaras de cílios
3	Boca	Batons, glosses e lápis labiais
4	Skincare	Cuidados com a pele, hidratantes e sérums

Figura 4. Tabela da entidade Categorias.

- **PRODUTOS:** Entidade central que guarda o estoque atual, preço de venda e as especificações de cada item de maquiagem.

123 id_produto	123 id_categoria	A-Z nome	A-Z marca	123 preco	123 quantidade_estoque
1	1	Base Matte HD	Boca Rosa	59,9	64
2	1	Corretivo Líquido	Tracta	29,9	102
3	1	Blush Péssago	Mari Maria	45	25
4	2	Máscara de Cílios	Maybelline	49,9	72

Figura 5. Tabela da entidade Produtos.

- FORNECEDORES: Registra as empresas parceiras que abastecem a loja, essencial para o controle de custos.

123 id_fornecedor	AZ razao_social	AZ cnpj	AZ telefone
1	Beauty Atacado Distribuidora	11.111.111/0001-11	1144444444
2	Glow MakeUp Fábrica	22.222.222/0001-22	1133333333
3	Skincare & Cia Importadora	33.333.333/0001-33	1122222222

Figura 6. Tabela da entidade Fornecedores.

- VENDAS: Cabeçalho de cada transação realizada com o cliente, contendo a data e o valor total.

123 id_venda	123 id_cliente	data_hora	123 valor_total
1	1	2026-06-15 12:43:30	318,9
2	2	2026-06-15 12:43:30	362,1
3	3	2026-06-15 12:43:30	418,1
4	4	2026-06-15 12:43:30	427,1

Figura 7. Tabela da entidade Vendas.

- ITENS_VENDA: Tabela associativa que detalha quais produtos e quantas unidades foram vendidas em cada operação de venda.

123 id_item_venda	123 id_venda	123 id_produto	123 quant	123 preco_unitario
1	1	1	1	59,9
2	1	2	2	29,9
3	1	3	1	45
4	1	4	1	49,9

Figura 8. Tabela da entidade Itens_Venda.

- PEDIDOS_COMPRA: Registra quando a loja solicitou reposição de estoque a um fornecedor específico.

123 id_pedido_compra	123 id_fornecedor	data_pedido	123 valor_total_custo
1	1	2026-06-15 12:43:30	2.930
2	2	2026-06-15 12:43:30	2.750
3	3	2026-06-15 12:43:30	3.250
4	1	2026-06-15 12:43:30	1.200

Figura 9. Tabela da entidade Pedidos_Compra.

- ITENS_PEDIDO_COMPRA: Detalha quais produtos foram comprados do fornecedor em cada pedido, registrando o preço de custo unitário.

123 id_item_pedido	123 id_pedido_compra	123 id_produto	123 quantidade_comprada	123 preco_custo_unitario
1	1	1	50	25
2	1	2	80	12
3	1	3	40	18
4	2	4	60	20

Figura 10. Tabela da entidade Itens_Pedido_Compra.

4. Implementação do Banco de Dados

Esta seção detalha o script de criação da estrutura do banco de dados loja_maquiagem, desenvolvido para o ambiente MySQL. O processo de implementação foi estruturado respeitando as dependências entre as tabelas (chaves estrangeiras), garantindo a integridade referencial do sistema. O código SQL para utilização está no mesmo .zip em que esse documento se encontra, no qual está dentro da pasta construtor_banco, em que, o da presente seção é o documento “gerador_banco.sql”, e o da próxima sessão será o “popular_banco.sql”.

4.1 Roteiro de Criação

O script foi organizado em uma sequência lógica, onde as entidades "pais" (tabelas independentes) são criadas antes das entidades "filhas" (tabelas dependentes). A utilização da instrução AUTO_INCREMENT foi adotada como prática padrão para a geração de identificadores únicos (Primary Keys), simplificando a inserção de registros e a manutenção das relações.

```
CREATE DATABASE IF NOT EXISTS loja_maquiagem;
use loja_maquiagem;

-- cria a tabela de Categorias (ind)
CREATE TABLE Categorias(
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    descricao TEXT
);

-- cria a tabela de Clientes (ind)
CREATE TABLE Clientes(
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    telefone VARCHAR(20)
);

-- cria a tabela de Fornecedores (ind)
CREATE TABLE Fornecedores (
    id_fornecedor INT AUTO_INCREMENT PRIMARY KEY,
    razao_social VARCHAR(150) NOT NULL,
    cnpj VARCHAR(20) UNIQUE NOT NULL,
    telefone VARCHAR(20)
);
```


-- cria a tabela de Produtos (depende de Categorias)

```
CREATE TABLE Produtos(  
    id_produto INT AUTO_INCREMENT PRIMARY KEY,  
    id_categoria INT NOT NULL,  
    nome VARCHAR(100) NOT NULL,  
    marca VARCHAR(50),  
    preco DECIMAL(10, 2) NOT NULL,  
    quantidade_estoque INT DEFAULT 0,  
    FOREIGN KEY (id_categoria) REFERENCES  
    Categorias(id_categoria)  
);
```

-- cria a tabela de Vendas (depende de Clientes)

```
CREATE TABLE Vendas(  
    id_venda INT AUTO_INCREMENT PRIMARY KEY,  
    id_cliente INT NOT NULL,  
    data_hora DATETIME DEFAULT CURRENT_TIMESTAMP,  
    valor_total DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)  
);
```

-- cria a tabela de Pedidos de Compra (depende de Fornecedores)

```
CREATE TABLE Pedidos_Compra (  
    id_pedido_compra INT AUTO_INCREMENT PRIMARY KEY,  
    id_fornecedor INT NOT NULL,  
    data_pedido DATETIME DEFAULT CURRENT_TIMESTAMP,  
    valor_total_custo DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (id_fornecedor) REFERENCES  
    Fornecedores(id_fornecedor)  
);
```

-- cria a tabela de Itens_Venda (depende de Vendas e Produtos)

```
CREATE TABLE Itens_Venda(  
    id_item_venda INT AUTO_INCREMENT PRIMARY KEY,  
    id_venda INT NOT NULL,  
    id_produto INT NOT NULL,  
    quantidade INT NOT NULL,  
    preco_unitario DECIMAL(10, 2) NOT NULL,  
    FOREIGN KEY (id_venda) REFERENCES Vendas(id_venda),  
    FOREIGN KEY (id_produto) REFERENCES Produtos(id_produto)  
);
```

```
-- cria a tabela de Itens do Pedido (depende de Pedidos_Compra e Produtos)
CREATE TABLE Itens_Pedido_Compra (
    id_item_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido_compra INT NOT NULL,
    id_produto INT NOT NULL,
    quantidade_comprada INT NOT NULL,
    preco_custo_unitario DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (id_pedido_compra) REFERENCES
    Pedidos_Compra(id_pedido_compra),
    FOREIGN KEY (id_produto) REFERENCES Produtos(id_produto)
);
```

5. População do Banco de Dados e Automação de Cálculos

A etapa de população do banco de dados visa garantir que o ambiente de testes reflita cenários reais de operação comercial. Foram inseridos registros nas tabelas de categorias, clientes, produtos e fornecedores, seguidos pelos processos de movimentação de estoque (pedidos de compra) e comercialização (vendas).

5.1 Integridade e Lógica de Dados

A inserção dos dados foi realizada seguindo a ordem de precedência das chaves estrangeiras. Destaca-se que o estoque dos produtos foi mantido inicialmente como zero, sendo este calculado dinamicamente através de procedimentos de automação que processam o fluxo de entrada (compras) e saída (vendas) de mercadorias.

O registro de uma nova operação comercial segue o padrão estrutural Mestre-Detalhe. Dessa forma, a aplicação deve primeiro realizar a inserção na entidade cabeçalho (como Vendas ou Pedidos_Compra), gerando o identificador da transação, e em seguida utilizar este identificador para inserir os múltiplos registros nas entidades associativas (Itens_Venda ou Itens_Pedido_Compra).

5.2 Triggers

Como os diagramas conceituais e lógicos (DER e IDEF1X) representam apenas a arquitetura estrutural dos dados, as regras de negócio ativas e as automações de cálculo foram implementadas diretamente no modelo físico através de Triggers (Gatilhos) no MySQL.

Essa abordagem garante que as regras de negócio fiquem encapsuladas no próprio banco de dados, reduzindo a latência e assegurando a consistência das informações independentemente da aplicação que estiver consumindo os dados. Foram desenvolvidas duas automações principais para o controle de auditoria:

5.2.1 Trigger de Auditoria de Entrada

Automatiza o recebimento de mercadorias. Ao registrar a compra de um item, o sistema atualiza dinamicamente o inventário e o custo consolidado do pedido.

DELIMITER \$\$

```
CREATE TRIGGER trg_auditoria_entrada_estoque
AFTER INSERT ON Itens_Pedido_Compra
FOR EACH ROW
BEGIN
    -- Incrementa o saldo do produto no estoque físico da loja
    UPDATE Produtos
    SET quantidade_estoque = quantidade_estoque +
    NEW.quantidade_comprada
    WHERE id_produto = NEW.id_produto;

    -- Soma o subtotal do item ao valor total devido ao fornecedor
    UPDATE Pedidos_Compra
    SET valor_total_custo = valor_total_custo +
    (NEW.quantidade_comprada * NEW.preco_custo_unitario)
    WHERE id_pedido_compra = NEW.id_pedido_compra;
END$$
```

DELIMITER ;

5.2.2 Trigger de Auditoria de Saída

Automatiza a operação de caixa. Ao registrar a venda de um item, o sistema realiza a baixa automática no inventário e calcula o faturamento da transação.

DELIMITER \$\$

```
CREATE TRIGGER trg_auditoria_saida_estoque
AFTER INSERT ON Itens_Venda
FOR EACH ROW
BEGIN
    -- Realiza a baixa da quantidade vendida do estoque da loja
    UPDATE Produtos
    SET quantidade_estoque = quantidade_estoque - NEW.quantidade
    WHERE id_produto = NEW.id_produto;
```

```

-- Soma o subtotal do item vendido ao valor total pago pelo cliente
UPDATE Vendas
SET valor_total = valor_total + (NEW.quantidade *
NEW.preco_unitario)
WHERE id_venda = NEW.id_venda;
END$$

DELIMITER ;

```

5.2.1 Auditoria de Entrada

Sempre que a loja recebe um novo lote de um fornecedor (Itens_Pedido_Compra), o banco de dados automaticamente incrementa a quantidade do produto no estoque e consolida o valor de custo no pedido.

```

DELIMITER $$

CREATE TRIGGER trg_auditoria_entrada_estoque
AFTER INSERT ON Itens_Pedido_Compra
FOR EACH ROW
BEGIN
    -- Incrementa o estoque da loja
    UPDATE Produtos
    SET quantidade_estoque = quantidade_estoque +
    NEW.quantidade_comprada
    WHERE id_produto = NEW.id_produto;

    -- Atualiza o custo total do pedido
    UPDATE Pedidos_Compra
    SET valor_total_custo = valor_total_custo +
    (NEW.quantidade_comprada * NEW.preco_custo_unitario)
    WHERE id_pedido_compra = NEW.id_pedido_compra;
END$$

DELIMITER ;

```

5.2.2 Auditoria de Saída

Ao registrar a venda de um item (Itens_Venda), o sistema realiza a baixa automática da quantidade no estoque da tabela Produtos e calcula o somatório do faturamento direto na tabela.

```
DELIMITER $$
```

```
CREATE TRIGGER trg_auditoria_saida_estoque
```

```
AFTER INSERT ON Itens_Venda
```

```
FOR EACH ROW
```

```
BEGIN
```

```
-- Realiza a baixa no estoque da loja
```

```
UPDATE Produtos
```

```
SET quantidade_estoque = quantidade_estoque - NEW.quantidade
```

```
WHERE id_produto = NEW.id_produto;
```

```
-- Atualiza o valor total da venda
```

```
UPDATE Vendas
```

```
SET valor_total = valor_total + (NEW.quantidade*NEW.preco_unitario)
```

```
WHERE id_venda = NEW.id_venda;
```

```
END$$
```

```
DELIMITER ;
```

5.3 Scripts de Carga e Processamento

Graças à implementação das Triggers documentadas na seção anterior, a inserção dos registros de comercialização já dispara automaticamente o recálculo do estoque e das métricas financeiras em tempo real, dispensando a execução de procedimentos de atualização (UPDATE) em lote. Abaixo, o script completo de carga de dados:

```
USE loja_maquiagem;
```

```
INSERT INTO Categorias (nome, descricao) VALUES
('Rosto', 'Bases, corretivos, pós e blushes'),
('Olhos', 'Sombras, delineadores e máscaras de cílios'),
('Boca', 'Batons, glosses e lápis labiais'),
('Skincare', 'Cuidados com a pele, hidratantes e sérums'),
('Acessórios', 'Pincéis, esponjas e curvex');
```

```
INSERT INTO Clientes (nome, email, telefone) VALUES
('Ana Silva', 'ana@email.com', '11999999999'),
('Beatriz Costa', 'bia@email.com', '11888888888'),
('Carolina Mendes', 'carol@email.com', '11777777777'),
('Daniela Souza', 'dani@email.com', '11666666666'),
('Fernanda Lima', 'fer@email.com', '11555555555');
```

```
-- produtos zerados inicialmente - nao compramos ainda rs
```

```
INSERT INTO Produtos (id_categoria, nome, marca, preco, quantidade_estoque)
VALUES
```

```
(1, 'Base Matte HD', 'Boca Rosa', 59.90, 0),      -- ID 1 -> categoria de Rosto
(1, 'Corretivo Líquido', 'Tracta', 29.90, 0),      -- ID 2 -> categoria de Rosto
(1, 'Blush Pêssego', 'Mari Maria', 45.00, 0),      -- ID 3 -> categoria de Rosto
(2, 'Máscara de Cílios', 'Maybelline', 49.90, 0),  -- ID 4 -> categoria de Olhos
(2, 'Delineador Caneta', 'Bruna Tavares', 42.50, 0), -- ID 5 -> categoria de Olhos
(3, 'Batom Líquido', 'Ruby Rose', 19.90, 0),      -- ID 6 -> categoria de Boca
(3, 'Gloss Labial', 'Vult', 22.00, 0),            -- ID 7 -> categoria de Boca
(4, 'Sérum Vitamina C', 'Principia', 59.00, 0),    -- ID 8 -> categoria de Skincare
(4, 'Hidratante Facial', 'Cerave', 89.90, 0),      -- ID 9 -> categoria de Skincare
(5, 'Esponja Gota', 'Mariana Saad', 35.00, 0);    -- ID 10 -> categoria de Acessórios
```

```
INSERT INTO Fornecedores (razao_social, cnpj, telefone) VALUES
('Beauty Atacado Distribuidora', '11.111.111/0001-11', '11444444444'),
('Glow MakeUp Fábrica', '22.222.222/0001-22', '11333333333'),
('Skincare & Cia Importadora', '33.333.333/0001-33', '11222222222');
```

INSERT INTO Pedidos_Compra (id_fornecedor) VALUES

(1), -- Pedido 1 (Beauty Atacado)

(2), -- Pedido 2 (Glow MakeUp)

(3), -- Pedido 3 (Skincare & Cia)

(1), -- Pedido 4 (Beauty Atacado)

(2); -- Pedido 5 (Glow MakeUp)

INSERT INTO Itens_Pedido_Compra (id_pedido_compra, id_produto, quantidade_comprada, preco_custo_unitario) VALUES

-- Pedido 1 (Comprando produtos de Rosto)

(1, 1, 50, 25.00), -- Base Matte

(1, 2, 80, 12.00), -- Corretivo Líquido

(1, 3, 40, 18.00), -- Blush Pêssego

-- Pedido 2 (Comprando produtos de Olhos e Boca)

(2, 4, 60, 20.00), -- Máscara de Cílios

(2, 5, 50, 15.00), -- Delineador Caneta

(2, 6, 100, 8.00), -- Batom Líquido

-- Pedido 3 (Comprando Skincare e Acessórios)

(3, 8, 30, 25.00), -- Sérum Vit C

(3, 9, 25, 40.00), -- Hidratante Facial

(3, 10, 150, 10.00),-- Esponja Gota

-- Pedido 4 (Reabastecendo os mais vendidos)

(4, 1, 30, 25.00), -- Base Matte

(4, 7, 50, 9.00), -- Gloss Labial

-- Pedido 5 (Mix final)

(5, 2, 40, 12.00), -- Corretivo Líquido

(5, 4, 30, 20.00), -- Máscara de Cílios

(5, 10, 100, 10.00);-- Esponja Gota

INSERT INTO Vendas (id_cliente) VALUES

(1), (2), (3), (4), (5), -- Vendas 1 a 5

(1), (2), (3), (4), (5), -- Vendas 6 a 10

(1), (2), (3), (4), (5), -- Vendas 11 a 15

(1), (2), (3), (4), (5); -- Vendas 16 a 20


```
INSERT INTO Itens_Venda (id_venda, id_produto, quantidade, preco_unitario)
VALUES
```

```
-- Venda 1
```

```
(1, 1, 1, 59.90), (1, 2, 2, 29.90), (1, 3, 1, 45.00), (1, 4, 1, 49.90), (1, 5, 1, 42.50),
(1, 6, 2, 19.90), (1, 7, 1, 22.00),
```

```
-- Venda 2
```

```
(2, 2, 1, 29.90), (2, 3, 1, 45.00), (2, 4, 2, 49.90), (2, 5, 1, 42.50), (2, 6, 1, 19.90),
(2, 7, 3, 22.00), (2, 8, 1, 59.00),
```

```
-- Venda 3
```

```
(3, 3, 1, 45.00), (3, 4, 1, 49.90), (3, 5, 1, 42.50), (3, 6, 1, 19.90), (3, 7, 1, 22.00),
(3, 8, 1, 59.00), (3, 9, 2, 89.90),
```

```
-- Venda 4
```

```
(4, 4, 2, 49.90), (4, 5, 1, 42.50), (4, 6, 1, 19.90), (4, 7, 1, 22.00), (4, 8, 2, 59.00),
(4, 9, 1, 89.90), (4, 10, 1, 35.00),
```

```
-- Venda 5
```

```
(5, 5, 1, 42.50), (5, 6, 1, 19.90), (5, 7, 2, 22.00), (5, 8, 1, 59.00), (5, 9, 1, 89.90),
(5, 10, 3, 35.00), (5, 1, 1, 59.90),
```

```
-- Venda 6
```

```
(6, 6, 1, 19.90), (6, 7, 1, 22.00), (6, 8, 1, 59.00), (6, 9, 1, 89.90), (6, 10, 1, 35.00),
(6, 1, 1, 59.90), (6, 2, 2, 29.90),
```

```
-- Venda 7
```

```
(7, 7, 2, 22.00), (7, 8, 1, 59.00), (7, 9, 1, 89.90), (7, 10, 1, 35.00), (7, 1, 2, 59.90),
(7, 2, 1, 29.90), (7, 3, 1, 45.00),
```

```
-- Venda 8
```

```
(8, 8, 1, 59.00), (8, 9, 1, 89.90), (8, 10, 2, 35.00), (8, 1, 1, 59.90), (8, 2, 1, 29.90),
(8, 3, 1, 45.00), (8, 4, 1, 49.90),
```

```
-- Venda 9
```

```
(9, 9, 1, 89.90), (9, 10, 1, 35.00), (9, 1, 1, 59.90), (9, 2, 2, 29.90), (9, 3, 1, 45.00),
(9, 4, 2, 49.90), (9, 5, 1, 42.50),
```

```
-- Venda 10
```

```
(10, 10, 1, 35.00), (10, 1, 1, 59.90), (10, 2, 1, 29.90), (10, 3, 1, 45.00),
(10, 4, 1, 49.90), (10, 5, 1, 42.50), (10, 6, 3, 19.90),
```

```
-- Venda 11
```

```
(11, 1, 2, 59.90), (11, 3, 1, 45.00), (11, 5, 1, 42.50), (11, 7, 1, 22.00), (11, 9, 1, 89.90),
(11, 2, 1, 29.90), (11, 4, 1, 49.90),
```

```
-- Venda 12
```

```
(12, 2, 1, 29.90), (12, 4, 2, 49.90), (12, 6, 1, 19.90), (12, 8, 1, 59.00),
(12, 10, 1, 35.00), (12, 1, 1, 59.90), (12, 3, 1, 45.00),
```

```
-- Venda 13
```

```
(13, 3, 1, 45.00), (13, 5, 1, 42.50), (13, 7, 2, 22.00), (13, 9, 1, 89.90), (13, 2, 1, 29.90),
(13, 4, 1, 49.90), (13, 6, 1, 19.90),
```

```
-- Venda 14
```

```
(14, 4, 1, 49.90), (14, 6, 2, 19.90), (14, 8, 1, 59.00), (14, 10, 1, 35.00),
(14, 3, 1, 45.00), (14, 5, 1, 42.50), (14, 7, 1, 22.00),
```

```
-- Venda 15
```

```
(15, 5, 1, 42.50), (15, 7, 1, 22.00), (15, 9, 2, 89.90), (15, 1, 1, 59.90), (15, 4, 1, 49.90),
(15, 6, 1, 19.90), (15, 8, 1, 59.00),
```

-- Venda 16

(16, 6, 1, 19.90), (16, 8, 1, 59.00), (16, 10, 1, 35.00), (16, 2, 2, 29.90),
(16, 5, 1, 42.50), (16, 7, 1, 22.00), (16, 9, 1, 89.90),

-- Venda 17

(17, 7, 2, 22.00), (17, 9, 1, 89.90), (17, 1, 1, 59.90), (17, 3, 1, 45.00), (17, 6, 1, 19.90),
(17, 8, 1, 59.00), (17, 10, 1, 35.00),

-- Venda 18

(18, 8, 1, 59.00), (18, 10, 2, 35.00), (18, 2, 1, 29.90), (18, 4, 1, 49.90),
(18, 7, 1, 22.00), (18, 9, 1, 89.90), (18, 1, 1, 59.90),

-- Venda 19

(19, 9, 1, 89.90), (19, 1, 1, 59.90), (19, 3, 2, 45.00), (19, 5, 1, 42.50), (19, 8, 1, 59.00),
(19, 10, 1, 35.00), (19, 2, 1, 29.90),

-- Venda 20

(20, 10, 1, 35.00), (20, 2, 1, 29.90), (20, 4, 1, 49.90), (20, 6, 2, 19.90),
(20, 9, 1, 89.90), (20, 1, 1, 59.90), (20, 3, 1, 45.00);

6. Consultas Analíticas e Validação do Sistema

Para validar a integridade dos dados, a eficiência das Triggers e demonstrar a capacidade analítica do banco de dados, foram desenvolvidas consultas gerenciais em SQL (DQL - Data Query Language). As queries foram divididas em três frentes de negócios: relatórios comerciais (B2C), gestão de fornecedores (B2B) e métricas financeiras de auditoria.

6.1 Relatórios Comerciais

6.1.1 Recibo Detalhado de Venda:

Esta consulta simula a emissão de um cupom fiscal, unindo dados do cliente, cabeçalho da venda e os itens comprados, calculando o subtotal da operação.

```
SELECT
    v.id_venda,
    c.nome AS cliente,
    p.nome AS produto,
    iv.quantidade,
    iv.preco_unitario,
    (iv.quantidade * iv.preco_unitario) AS subtotal
FROM Vendas v
JOIN Clientes c ON v.id_cliente = c.id_cliente
JOIN Itens_Venda iv ON v.id_venda = iv.id_venda
JOIN Produtos p ON iv.id_produto = p.id_produto
WHERE iv.id_venda = 1;
```

6.1.2 Curva ABC de Clientes (Maior Volume de Compras):

Agrupa as vendas por cliente, retornando a quantidade de pedidos e o Lifetime Value (Total Gasto), ordenando pelos consumidores mais rentáveis.

```
SELECT
    c.nome AS cliente,
    COUNT(v.id_venda) AS quantidade_de_compras,
    SUM(v.valor_total) AS total_gasto
FROM Clientes c
JOIN Vendas v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente
ORDER BY total_gasto DESC;
```

6.1.3 Produtos Mais Vendidos e Faturamento Total:

Retorna o desempenho do catálogo de produtos e o somatório bruto de todas as vendas registradas no sistema.

-- Produtos mais vendidos

```
SELECT
    p.nome AS produto,
    cat.nome AS categoria,
    SUM(iv.quantidade) AS total_unidades_vendidas
FROM Produtos p
JOIN Itens_Venda iv ON p.id_produto = iv.id_produto
JOIN Categorias cat ON p.id_categoria = cat.id_categoria
GROUP BY p.id_produto
ORDER BY total_unidades_vendidas DESC;
```

-- Faturamento total bruto

```
SELECT
    COUNT(id_venda) AS total_de_pedidos,
    SUM(valor_total) AS faturamento_total_bruto
FROM Vendas;
```

6.2 Gestão de Fornecedores

6.2.1 Espelho de Pedido de Compra e Investimento por Fornecedor:

As consultas abaixo permitem rastrear exatamente o que foi comprado em um lote específico e qual fornecedor recebe a maior fatia do orçamento da loja.

-- Recibo de entrada de mercadoria (Exemplo: Compra 1)

```
SELECT
    pc.id_pedido_compra,
    f.razao_social AS fornecedor,
    p.nome AS produto,
    ipc.quantidade_comprada AS qtd,
    ipc.preco_custo_unitario AS custo_unitario,
    (ipc.quantidade_comprada * ipc.preco_custo_unitario) AS subtotal_custo
FROM Pedidos_Compra pc
JOIN Fornecedores f ON pc.id_fornecedor = f.id_fornecedor
JOIN Itens_Pedido_Compra ipc ON pc.id_pedido_compra =
ipc.id_pedido_compra
JOIN Produtos p ON ipc.id_produto = p.id_produto
WHERE pc.id_pedido_compra = 1;
```

```
-- Curva ABC de Fornecedores
SELECT
    f.razao_social AS fornecedor,
    COUNT(pc.id_pedido_compra) AS total_de_pedidos,
    SUM(pc.valor_total_custo) AS investimento_total
FROM Fornecedores f
JOIN Pedidos_Compra pc ON f.id_fornecedor = pc.id_fornecedor
GROUP BY f.id_fornecedor
ORDER BY investimento_total DESC;
```

6.3. Métricas Financeiras e Auditoria de Estoque

6.3.1 Análise de Margem de Lucro por Produto:

Cruza o maior preço pago no fornecedor com o preço de venda na loja, estabelecendo o lucro bruto unitário.

```
SELECT
    p.nome AS produto,
    cat.nome AS categoria,
    MAX(ipc.preco_custo_unitario) AS custo_no_fornecedor,
    p.preco AS preco_de_venda_na_loja,
    (p.preco - MAX(ipc.preco_custo_unitario)) AS lucro_bruto_por_unidade
FROM Produtos p
JOIN Categorias cat ON p.id_categoria = cat.id_categoria
JOIN Itens_Pedido_Compra ipc ON p.id_produto = ipc.id_produto
GROUP BY p.id_produto
ORDER BY lucro_bruto_por_unidade DESC;
```

6.3.2 Auditoria Automatizada de Inventário:

Esta consulta atesta o funcionamento das Triggers. Ela compara o saldo dinâmico do estoque com o histórico de compras, utilizando estrutura condicional (CASE WHEN) para alertar sobre movimentações ou anomalias.

```
SELECT
    p.nome AS produto,
    p.quantidade_estoque AS estoque_atual_na_loja,
    SUM(ipc.quantidade_comprada) AS total_comprado_do_fornecedor,
    CASE
        WHEN p.quantidade_estoque < SUM(ipc.quantidade_comprada)
        THEN 'Menor (Vendas ocorreram)'
        WHEN p.quantidade_estoque > SUM(ipc.quantidade_comprada)
        THEN 'Maior (Erro/Anomalia)'
        ELSE 'Igual (Nenhuma unidade vendida)'
    END AS status_comparacao
```

```
FROM Produtos p
JOIN Itens_Pedido_Compra ipc ON p.id_produto = ipc.id_produto
GROUP BY p.id_produto
ORDER BY p.nome;
```

7. Conclusão

O desenvolvimento deste projeto permitiu a aplicação prática e aprofundada dos conceitos de modelagem e implementação de bancos de dados relacionais. A criação do sistema para a loja de maquiagens atendeu com êxito aos requisitos de negócio propostos, englobando e rastreando todo o ciclo de vida das mercadorias, desde a aquisição junto aos fornecedores até a comercialização para o consumidor final.

A fase de modelagem, suportada pelos diagramas conceitual (DER) e lógico (IDEF1X), demonstrou-se crucial para garantir a normalização dos dados e o estabelecimento de uma arquitetura coesa e livre de anomalias de inserção ou deleção. O planejamento cuidadoso da integridade referencial e a adoção consciente do padrão Mestre-Detalhe para os registros transacionais criaram uma base de dados altamente confiável.

O grande diferencial desta implementação foi a adoção de automações nativas no modelo físico através de *Triggers*. Ao encapsular as regras de negócio de fluxo de caixa e controle de inventário diretamente no MySQL, o sistema garante uma auditoria em tempo real e elimina o risco de inconsistências de dados, apresentando uma latência mínima e um nível de segurança compatível com aplicações corporativas.

Por fim, a bateria de testes baseada em consultas analíticas (DQL) atestou a eficiência do modelo estruturado. Foi possível comprovar que o banco de dados desenvolvido transcende a função de um mero repositório de registros, atuando como uma verdadeira ferramenta de *Business Intelligence*, capaz de fornecer relatórios gerenciais, curvas ABC e métricas financeiras essenciais para a tomada de decisão. O projeto finaliza-se como uma solução escalável, íntegra e perfeitamente alinhada com as melhores práticas da engenharia de dados.

8. Referências Bibliográficas

- 1) CARVALHO, V. MySQL: Comece com o principal banco de dados open source do mercado. São Paulo: Casa do Código, 2015.
- 2) ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 6. ed. São Paulo: Pearson Addison Wesley, 2011.
- 3) MYSQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 10 jun. 2026.
- 4) PLANTUML. Open-source tool that uses simple textual descriptions to draw UML diagrams. Disponível em: <https://plantuml.com/pt/>. Acesso em: 11 jun. 2026.
- 5) SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Sistema de banco de dados. 6. ed. Rio de Janeiro: Elsevier, 2012.